Dresden
University of
Technology

A smart meter MCU emulation, introducing ACAE: An ARM cycle accounting emulator

Jakob Fregin

June 2026

Smart meters are increasingly considered as potential edge devices in future smart grids, enabling more advanced local processing beyond simple measurement and reporting. However, research on extended smart meter functionality is limited by the proprietary nature of deployed hardware platforms and the lack of open development and evaluation environments. This makes it difficult to prototype, validate, and assess the feasibility of novel smart meter applications before deployment.

In this work, we present ACAE, an ARM cycle-accounting emulator for smart meter microcontroller units. ACAE is built upon a representative smart meter microcontroller unit (MCU) model derived from publicly available specifications and implemented as an extended ARM Cortex-M4 board within the QEMU emulation platform. The emulator incorporates cycle-accounting capabilities to enable performance-oriented evaluation of software running in a hardware-independent setting.

We evaluate ACAE using the Dhrystone and CoreMark benchmarks and compare results against representative real hardware. Our evaluation shows that Dhrystone executes 74.6% faster in the emulator, while CoreMark executes 51.4% faster. Despite these absolute differences, ACAE reproduces key performance trends observed on hardware, indicating that the cycle-accounting model captures relevant execution behavior. These results suggest that ACAE provides a promising foundation for estimating performance characteristics of smart meter applications.

Overall, ACAE enables early-stage exploration of smart meter software and facilitates research into extended smart grid functionality, while highlighting the need for further refinement of cycle-accurate modeling and broader validation across workloads and architectures.

1 Introduction

As resource distribution systems, such as electricity grids, become increasingly complex due to growing demand and distributed generation, traditional grid management faces significant challenges. Smart grids (SGs) provide the digital infrastructure required to modernize resource networks by enabling real-time monitoring and communication between consumers and providers. This advanced metering infrastructure (AMI) provides an enhanced view of the network, allowing for more resource-efficient distribution through supply-demand matching, reducing waste of energy and improving grid stability [1]. Furthermore, the data collected by smart meters (SMs) supports extended applications such as outage prediction, faster fault detection, remote management, and prioritization in emergency situations [2]. Altogether, these components can contribute to a more reliable, resilient, and sustainable resource distribution system.

In its current state as of 2026 in Germany, the regulatory framework primarily treats SMs as trusted measurement and reporting devices [3, 4], while additional processing is performed at different points in the system. However, SMs pose significant challenges to consumer privacy. Due to their real-time operation and high sampling rates, they collect large amounts of potentially sensitive information [5]. Previous work has shown that this data can reveal the presence of occupants in a household as well as indicators of their socioeconomic status based on consumption patterns [5]. Furthermore, detailed analysis of SM measurements can infer the usage of individual appliances [6], exposing consumers' habits and daily routines [6]. These concerns motivate the execution of privacy-preserving data processing techniques directly on the device, allowing sensitive consumption data to be anonymized, aggregated, or otherwise transformed before transmission.

We propose that future SG applications may require SMs to perform additional local computation. However, developing and evaluating software for SMs remains challenging. The software stacks and hardware (HW) architectures of commercially deployed devices are often proprietary and inaccessible to researchers and third-party developers. Furthermore, standardized software development kits, testing environments, and hardware emulators are generally unavailable. As a result, experimentation with novel SM applications is difficult, limiting the ability to prototype, validate, and assess the feasibility of new approaches before investing in implementation on production hardware. These limitations motivate the need for open and reproducible development environments that facilitate research on next-generation SM applications.

To address this challenge, we introduce an ARM cycle accounting emulator (ACAE) designed to provide a HW-independent platform for development and evaluation of extended SM functionality. The emulator aims to provide a software-based simulation with sufficient temporal resolution to estimate the performance impact of newly proposed algorithms and services. In this way, ACAE aims to enable rapid prototyping and comparative evaluation of SM applications without requiring access to proprietary devices.

To realize ACAE, we construct a representative SM MCU emulator. First, we investigate the HW characteristics of recent SM MCUs and derive a representative MCU model. Based on this model, we configure an existing emulation platform and extend it with a cycle-accounting mechanism. Finally, we validate the emulator against measurements obtained from representative HW and evaluate ACAEs' timing performance with respect to real HW.

The remainder of this paper is structured as follows. Section 2 provides general background on SMs, the MCU of our model, and HW simulation. Section 3 describes the representative SM MCUs model, the cycle model, as well as the HW validation model. Section 4 explains our approach to emulation and validation. Section 5 evaluates our results, states limitations, and proposes potential for future work.

2 Background

In the following section, we present background information about essential underlying technologies, such as SMs, MCUs with special focus on the ARM Cortex-M4, which we assume in our model, as well as benchmarking.

2.1 Smart Meter

SMs are essential components of an SG, constituting the edge nodes of an AMI. The AMI is a network of sensors, aggregation, and analysis of collected data [2]. The SM is an advanced resource meter, monitoring fine-grained resource consumption directly at the end customer [7]. This provides a real-time view of the resource consumption for the customer [7].
SMs report their sensor values to smart meter gateways (SMGWs). SMGWs then provide the first aggregation. They receive real-time data from multiple geographically nearby SMs and forward the aggregate to the meter data management system (MDMS) [2]. The MDMS is the central entity (CE). It collects metrology data across the entire SG and performs the necessary analysis [2]. Additionally, the AMI implements bidirectional communication between a customer's SM and the resource provider [2]. This is done using a various communication standards [7, 1]. This centralized data and analysis, provided by the AMI extends the capabilities of the SG beyond a traditional grid. It allows for better supply demand matching, faster outage detection and restoration, as well as value-added services for the customers, such as providing their real-time consumption or dynamic pricing [2].
Selected from these components, SMs pose the weakest system in regard to compute resources. Since SMs are the most deployed and distributed, they are not equipped with more powerful processors as SMGWs are or reside in a data center setting as the CE. At its core, SMs consist of analog-to-digital converters (ADCs) for metrology, a real-time clock (RTC) for accurate timestamping, and a MCU running firmware to acquire and calculate metrology data. In terms of electricity SMs, these measurements include root-mean-square (RMS) voltage and current, active and reactive power, as well as power factor and frequency [2].

2.2 Microcontroller Unit

The MCU is a general-purpose computer executing its firmware stored in nonvolatile flash memory. Regarding firmware there are several types.
Bare-metal firmware is software that runs directly on the hardware and is therefore most tailored to a specific HW architecture. The entry point of execution is defined at a platform-dependent memory address from which the first instruction is fetched, decoded, and executed, post-incrementing the program counter [8]. This first instruction invokes the boot sequence. On bare-metal, this boot sequence is part of the user program, and all steps involved are compiled into a single monolithic binary. The boot sequence includes setting the initial stack pointer to the upper random access memory (RAM) address, implementing a downwards growing stack. Following that, the data segment containing the user program is copied from flash memory to RAM, the segment containing statically defined uninitialized variables is set to zero, and finally, a heap memory region is reserved [9]. This concludes a simple boot sequence, and the main function containing the user program is called. Furthermore, a bare-metal setup requires to provide some fundamental backend functions if the C standard library (STD) is used, providing primitives such as stdio or heap allocations.
Another option for firmware is the use of a real-time operating system (RT-OS), which, contrary to the bare-metal setup, handles boot and standard library provisioning for the user. It generally

provides some form of HW abstraction, not requiring the user program to interface with the HW directly.
Lastly, there are operating systems (OSs) allowing the execution of multiple user programs and the OS handling context switches, scheduling central processing unit (CPU) time, and abstracting fully from HW using drivers. This also impedes the largest overhead with significant time spent in kernel execution. In contrast, bare-metal executes the user program only until it exits and then halts.

2.2.1 The ARM Cortex-M4

For our emulator, we aim to choose an MCU that best reflects HW used in real-world SMs. We describe the derivation of the essential characteristics in Sec. 3.1.2 in detail. Based on our analysis, the ARM Cortex-M4 is a common off-the-shelf MCU that aligns best with the characteristics of chips used in SMs.
The following section is a summary of the ARM Cortex-M4 documentation [10], focusing on the most important properties for our work. The ARM Cortex-M4 is a chip standard implemented by different vendors, encompassing a chip family with a standardized architecture and optional extensions.
The ARM Cortex-M4 uses the ARMv7 Thumb and Thumb2 instruction set, being a 32-bit RISC architecture. It implements a 3-stage pipeline consisting of instruction fetch, decode, and execution and doing so simultaneously. This causes the execution of 1 instruction per clock cycle, given that no pipeline stalls occur and excluding heavier instructions, such as division. Other reasons for longer execution times are instruction fetches, loads, or stores from or to flash memory. Flash is slower than a CPU clock cycle and requires the implementation of wait states, stalling execution until the flash response time is matched. In our case, using a CPU frequency of 100MHz, 3 such wait states are required. To mitigate these stall cycles, an instruction cache, as well as a data cache, is used, requiring no extra cycles as long as the data remains cached.
Beyond that, the chip features a nested vectored interrupt controller (NVIC), multiple high-performance bus interfaces, as well as an optional memory protection unit (MPU) and floating-point unit (FPU) with IEEE 754 single precision compliance. Another important feature is the low-cost on-board debug solution, allowing for runtime performance instrumentation, which we use for time measurements. Explicitly, if debug mode is enabled, a cycle count register can be queried, allowing for accurate time measurements with respect to the chip's clock frequency.

2.3 HW Simulation

A HW simulation running on a host machine provides a guest environment replicating selected HW characteristics in software. This allows the execution of guest code on the host machine using this guest environment in place of real HW components.
In order to simulate a guest HW MCU on a host machine, there are several, already existing, simulators, such as QEMU [11], gem5 [12], or renode [13], amongst many others. When choosing a simulator for scientific research, there are several aspects to consider [14], but most important for our purposes is the distinction between functional and cycle-accurate simulation. Both types have significant advantages and drawbacks. Cycle-accurate simulators execute instructions by simulating the processor's operations for each cycle. Specifically, cycle-accurate simulators reproduce the exact data flow on the register-transfer-level (RTL) [14]. In contrast, functional simulators only replicate the functional behavior of the guest's instruction set architecture (ISA), disregarding any micro-architectural behavior [14]. This difference in

granularity causes emulation with cycle resolution to be generally significantly slower than functional emulators [14]. A major problem we faced with cycle-accurate simulators was the restricted access to vendor emulators or cycle-accurate models. While we model an ARM MCU and ARM does provide cycle-accurate models, so-called CPAKs, with a free educational license on request, handling and delivery delays of those resources did not fit our time frame. In addition to requiring a cycle model, a full in-depth hardware simulation would have gone far beyond the scope of this work.
In contrast, functional simulators or so-called emulators inherently cannot resolve time or micro-architectural effects and are fundamentally not fit for precise performance measurements. Functional simulators can be used for identifying a program's architectural behavior, revealing the instruction trace, containing its distribution of instruction types, but also memory access, allowing for analysis regarding locality [14]. QEMU, gem5s "SimpleAtomic" CPU model, and renode are functional emulators.
There is existing work [15] combining the best of both worlds. They provide a cycle-accurate timing model for their functional instruction-set-simulator (ISS) called JIT DBT engine, surpassing traditional HW FPGA-based simulators in speed. Their work poses a proper approach to fast and cycle-accurate simulation, but its complexity reaches far beyond the scope of our work.
Our final choice for fast emulation and attached instrumentation is QEMU [11] using its ability to load user-written plugins. QEMU is a functional emulator, dynamically translating guest binary instructions during execution [16]. This is done in so-called translation blocks (TBs), aggregating blocks of contiguous instructions without any context change, such as jump or branch instructions [16]. These TBs are then stored for immediate execution, as well as potential re-execution at another point in time. This creates two distinct events to instrument: The translation once per TB and the following TB execution. QEMU's plugin API provides these instrumentation points as callbacks that can be registered by the user, which is then integrated into the instruction flow [11]. Beyond that, QEMU provides the icount execution mode, resulting in a deterministic execution of instructions in the absence of asynchronous interrupts [11]. In contrast to gem5, QEMU supports the ARMv7 32bit Thumb/2 ISA, which is needed for our work, and QEMU's plugin interface provides a simpler deterministic instrumentation than renode.
Ultimately, we used QEMU for our emulator and provided an ARM Cortex-M4 style debug cycle count register through a slight modification of the QEMU implementation and update the register through an instrumentation plugin.

2.4 Benchmarking

In regard to performance validation of simulated or real-HW systems, benchmarking is necessary. There are several benchmarks for embedded systems [17, 18] and general sensor networks [19, 20, 21].
The most useful benchmarks for MCUs are Dhrystone and EEMBC's CoreMark, since their scores are directly provided by the MCU vendors in their respective data sheets, which can be seen in our data sheet collection [22].
Dhrystone is an integer-operation-based benchmark published in 1984 [23]. In the context of embedded systems, the Dhrystone benchmark is a general metric for MCU performance, whilst not being capable of representing modern workloads [17]. Nevertheless, it is widely used and also utilized in research [24]. While its simplicity allows Dhrystone to be easily ported to many platforms, it resides most of its runtime in STD functions, such as strcmp, strcpy, and memcpy. Therefore, it is heavily dependent on the platform implementation, as well as compiler optimizations [17].
While Dhrystone reports its score as Dhrystones per second, the general score is reported in Dhrystone mega instructions per second (DMIPS). This score in DMIPS is calculated by referencing Dhrystones per second against the baseline score of 1757 Dhrystones per second. This

baseline is a score originally measured on a VAX 11/780 machine [17].
A more modern benchmark for embedded systems is CoreMark by the embedded microprocessor benchmark consortium (EEMBC) [18]. It provides list, string, and matrix workloads, resulting in a single score, given by iterations per second, called CoreMark [25]. Contrary to Dhrystone, it has a strict rule set for valid benchmark results. These rules include which files are allowed to be changed and score formatting, including compiler version and arguments, as well as CoreMark version, iterations, and the score [18]. Additionally, the EEMBC provides publicly hosted scores, submitted by users, adhering to the reporting rules. As mentioned earlier, CoreMark has become a general metric for MCU performance and is found in some vendor data sheets.
Beyond those two benchmarks, there are other benchmarks [19, 20, 21] focusing on embedded sensor networks. Additionally, they measure the performance of smart home features, networking, and security aspects of embedded systems. While these benchmarks could provide relevant insights for an SM environment, our initial emulation does not feature the required peripherals. Because of that and scarce reference data, we will not use these benchmarks.
In this work, we will use Dhrystone and CoreMark to acquire performance data.

3 System Model

Our goal for the SM MCU model is to represent the general computational performance of a recent, but average SM. This shall serve as a foundation to simulate more complex operations like cryptographic operations in future analyses. In order to build such a model, we required extensive data on SM internal HW to acquire a comprehensive representation of the current SM market.

3.1 Smart Meter Model

3.1.1 Specification Acquisition

First, we searched for general SMs and their documentation. We found several vendors and their different SM solutions. These vendors generally documented the external properties of their SMs in data sheets. This external data often includes operating voltages, frequencies, temperatures, connecting cable specifications, current limits, communication interfaces, the accuracy of measurements, and deviations. While those specifications are sufficient for a contractor installing these devices, there are no mentions of internal properties, such as used MCUs, networking controllers, or metrology units. We then searched for different documentation that was publicly available, such as those intended for repair or maintenance, but those as well were unavailable.
Next, we considered contacting SM vendors directly. We researched the biggest SM vendors and their products, since these could provide the most accurate data for residential SMs that are being deployed in the real world. We found articles analyzing the market share of SM vendors and ranking them accordingly [26, 27]. Amongst them, we chose the recent article [27], mentioning the 10 most prevalent SM vendors in 2025. Hindered by contact formalities, we contacted 8, requesting any information about their SM implementations, stating our research goal and emphasizing our interest in explicit MCU specifications used in SMs. We did not receive any response.
Since a regulatory institute could not provide us with information either, we returned to searching for publicly available articles and data sheets.
We collected SM reference designs, articles describing SM architecture, and MCU data sheets, stating SMs as their application. The proposed application for smart metering indicates that

the chips are at least potential solutions for SMs, and, presumably, also deployed in real HW devices. To keep a recent, but sizeable dataset of specifications, we disregarded any data sheets from before 2010. This resulted in a dataset containing a SM reference design by Texas Instruments and 19 MCU data sheets, suggesting SMs as their application. We accumulated these chips and their specifications to then estimate the variety of chips used. The data shows that there are reoccurring chip families and most modern SM MCUs use a similar architecture. Based on this collection of SM MCU data sheets [22], we then derive our model of an SM MCU.

3.1.2 Deriving the Model

From our gathered chip specifications, we extracted metrics that were relevant for the emulation of our model, as well as shared between chips, contributing to a unified model.
Those metrics include:

- ISA
- architectural standards
- core count
- word size
- clock speed
- RAM size and type
- cache size
- flash size
- hardware extensions, such as:
  - floating point unit (FPU)
  - dedicated cryptography and hashing units (AES, SHA, HMAC)
  - analog to digital converters (ADC)

Furthermore, we collected additional metrics to evaluate the relevance of the chips. By analyzing, whether certain features are present or generally how feature-rich a chip is, we can estimate more accurately, whether a chip could be implemented in a current smart meter or a general sensor network node.
The metrics used for this meta information include:

- year of the data sheet
- communication standards supported by the chip
- ADC type, channels, sample rate, resolution
- Core family
- Chip family
- RAM size and type
- flash size

All those attributes were then collected in a table [22]. We then evaluated an attribute value for each attribute, based on the meta attributes, the attributes themselves, and their alignment with respect to the rest of the dataset. This value is meant to represent a recent average, with a tendency towards being future-proof. These attributes and values finally form our representative SM MCU model.
Given this model, we reached out to the regulatory institute once more to present our model. They were able to verify that our model was indeed representative of an average SM MCU, which we consider proof, regarding the correctness of our model, matching our goals.

3.1.3 Representative Smart Meter MCU Model

Table 1 shows our representative SM MCU model. The model is based on our collected data that was collected as described in Sec. 3.1.1.
We model the average SM MCU core as an ARM Cortex-M4 as described in 2.2.1, since this chip aligns best with the characteristics of our collected data. Importantly, our model considers two of such cores. One core is used for metrology, using a sigma-delta ADC, as well as a single precision FPU for accelerated calculation of the measurement results. The other core is used for communication of the measurements, as well as for general compute and control. This includes value-added features, such as presenting information on a display. Notably, the compute-core has an AES HW accelerator supporting various operation modes, which would be used for secure communication to the upstream SMGW.

Property	Value	Unit
Core standard	ARM Cortex M4	
Instruction Set Arch.	ARM Thumb®2	
Cores	2	
Word size	32	bit
CPU clock frequency	100	MHz
RAM	128	KB
RAM type	SRAM	
Cache	2	KB
Flash	512	KB
Compute Core (emulated)		
FPU	no	
AES (dedicated access)	yes	
AES words	128, 192, 256	bit
AES modes	GCM, CBC, ECB, CFB, CBC-MAC, CTR	
SHA (dedicated access)	no	
HMAC (dedicated access)	no	
Metrology Core (not emulated)		
ADC	yes	
ADC type	sigma-delta	
ADC channels	4	
ADC word	20	bit
FPU	yes	
FPU word	IEEE 754 single precision	

Table 1: MCU Model Specifications

3.2 Cycle Model

A core module of our cycle-accounting emulation is the cycle model. It translates executed instructions into an estimation of cycles that would have passed on a real HW MCU. The cycle model is a main component of our QEMU plugin, as it is queried whenever TB is translated, storing the associated cycle count of that TB. This cycle count is then accumulated every time the TB is executed.
The cycle model used in this work is a first rudimentary implementation, resolving instruction cycles and memory access cycles only. Instruction cycles are estimated by 1 cycle for all instructions, excluding load and store instructions which contribute a cycle count of 2. This matches the ARM Cortex-M4 documentation, as we explain in Sec. 2.2.1. This cycle count of 2 for load or store represents a rough estimate of the one cycle the instruction requires executing and one extra cycle until the value is available for the next instruction to be used. Regarding memory access cycles, we assume two different memory regions. The flash memory is accounted for by 1 additional memory access cycle, since most flash accesses will be cached, therefore will take no additional cycles, but cache misses will meet a penalty of 3 wait states, given our model. The SRAM memory region is clocked synchronously to the CPU and will return the requested value to the register within this single clock cycle, therefore no extra wait state is needed, and no additional cycle is attributed in our cycle model.
These assumptions for our current cycle model are not tuned to match real HW particularly well, but instead chosen for prototyping and a first proof of concept. This cycle model provides a lower bound to the cycles required for a particular instruction. While most instructions execute pipelined in 1 cycle, according to the ARM documentation, some heavier instructions, such as division or branches, will be underestimated. Further improvement of the cycle model will be needed for using the emulator as an accurate HW replacement for scientific measurements.

3.3 HW Model

The HW model used for validation of our SM emulation model matches as closely as practically feasible, as shown in Fig. 2. We use an STM32F411CEU6 on a BlackPill board from WeAct Studio, which is an off-the-shelf single-core ARM Cortex-M4 MCU. It has 512 KB of flash memory and 128 KB of SRAM. For its CPU clock, it supports an internal clock, as well as a 25MHz external crystal oscillator, which is populated on our board. In order to match our model of a 100 MHz CPU clock, the chip uses a phase-locked loop (PLL) system to multiply the high speed external (HSE) base clock. Given those specifications, it matches our SM MCU model sufficiently for accurate validation.
A limitation arises when using the native USB port on the board for benchmark result reporting. While the direct use of the on-board USB interface poses the most convenient setup for validation, it requires an exact clock of 48MHz and a USB stack implementation. Due to on-board clock distribution, this would cause an integer-scaled CPU clock frequency of 96MHz, not matching our model, as well as introduce overhead by the USB stack.
Considering these issues, we opted to use a primitive Universal Asynchronous Receiver-Transmitter (UART) interface on two of the chip's IO pins instead. This provides light-weight serial communication, which we use for benchmark results and general state reporting outside any critical code sections for time-sensitive measurements. In order to conveniently acquire the benchmark results, we used another chip that has an on-board serial-to-USB interface as a relay to then report any UART output directly into the serial console [22]. We note that this secondary chip is used for relaying of serial communication and therefore in no means impacting any performance measurements.

Property	Value	Unit
Core standard	ARM Cortex M4	
Instruction Set Arch.	ARM Thumb®2	
Cores	1	
Word	32	bit
CPU clock frequency	100	MHz
RAM	128	KB
RAM type	SRAM	
Cache	present	
Flash	512	KB
Compute Core (emulated)		
FPU	yes, but disabled	
AES (dedicated access)	no	
SHA (dedicated access)	no	
HMAC (dedicated access)	no	
Metrology Core (not emulated)	not populated	

Table 2: Black Pill Board (STM32F411CEU6) specifications

4 Methodology

4.1 Emulation

Following our Model, we searched for ARM-compatible freeware emulators, as mentioned in Sec. 2.3. Since we require a timing accurate emulator for performance benchmarking, the first choice would be a cycle-accurate emulator, which fully represents the MCU's components architecture, such as the pipeline, caches, and connecting buses. As we presented in Sec. 2.3, this would go beyond our scope, which is why we chose QEMU as our emulator.
QEMU tracks the guest execution state and translates all guest instructions on demand in as TBs. These TBs are kept in strict order. QEMU provides an execution mode called icount, which causes any guest instruction to be executed sequentially, allowing for individual instrumentation using QEMU's TCG plugins. This results in a functional and therefore faster than a cycle-accurate emulator that allows for precise instruction counting.
However, instruction counting does not resolve time, clock cycles do. In order to resolve HW cycles approximately, we introduced an instruction-to-cycle model, as described in Sec. 3.2. This cycle model is queried to account for any passing instruction during execution and convert the computation into an approximated real-world cycle count. This cycle count is then passed back into the guest environment at runtime to provide our cycle count register. The cycle count register can then be used by any guest application to get a real-time estimate. We note that the register is a 32-bit unsigned integer directly reporting cycles. This limits the maximum time measurement span to 2^{32} cycles, which we resolve by wrapping overflows. All of our implemented benchmarks were adjusted to use this cycle count for their time measurement. Altogether, this yields a cycle accounting emulator.

4.2 Validation

To validate our emulator, the selected benchmarks Dhrystone and CoreMark were executed both on our emulator and on real HW. For comparability, all benchmarks were compiled as bare-metal to monolithic binaries with identical configurations on both platforms. All benchmark runs were compiled without compiler optimizations and without debug information. The use of

a bare-metal environment combined with disabled optimizations was intended to maximize determinism and reproducibility across all measurements.
For the validation HW, the generated binaries were programmed onto the target device using its bootloader operating in DFU mode. Firmware transfer was performed from the host system using `dfu-util` via the on-board USB interface. After flashing, the device automatically reset and transitioned from DFU mode into normal execution mode. The benchmark binary then entered the boot sequence described in Sec. 2.2, executed the benchmark, and halted. Benchmark results were transmitted via UART, as described in Sec. 3.3 and collected.
The experimental setup required several modifications to both benchmarks. First, benchmark restrictions regarding minimum execution time were removed. This was necessary because of the limited range of the cycle count register impeding an upper limit on execution time. As a result, benchmark executions with low iteration counts became possible. These short executions contain a comparatively large proportion of non-benchmark overhead, which is the reason benchmark specifications normally impose restrictions on valid runs. While such runs are not representative of absolute benchmark performance and therefore partially invalidate benchmark-compliant score reporting, they provide additional reference points for comparing emulator and HW characteristics.
To ensure compatibility with the bare-metal setup, benchmark calls to selected STD and system functions were replaced with equivalent bare-metal implementations. Furthermore, an automated score readout mechanism was added to simplify the execution of multiple benchmark configurations in succession and to allow automatic result collection. No modifications were made to any time-sensitive benchmark sections. Consequently, all reported scores remain representative of the timing behavior of the respective execution platform.
The iteration count for each benchmark was specified through the `ITERATIONS` make variable, which was forwarded to the `arm-none-eabi-gcc` preprocessor and integrated into the benchmark build process. For all benchmark runs, compilation was performed using the `-O0` and `-g0` compiler flags, thereby disabling optimizations and excluding debug information.
The benchmark-specific configurations differed as follows. Dhrystone was compiled using the `-std=gnu89` flag, as it relies on C89 language features within its time-sensitive code sections, which were intentionally left unchanged. Due to the cycle count register overflow limitation described above, Dhrystone measurements were restricted to iteration counts ranging from 1 to 2^{20}. Following execution, Dhrystone scores were reported in DMIPS and DMIPS/MHz through our modifications. These values are derived from the benchmark's original output in Dhrystones per second by referencing the VAX 11/780 baseline as described in Sec. 2.4.
CoreMark, being a more recent benchmark, was compiled using the `arm-none-eabi-gcc` default C17 standard. Measurements were performed for iteration counts ranging from 1 to 2^{11}. After each execution, CoreMark reported its score in iterations per second. These values were subsequently presented as scores in CoreMark and CoreMark/MHz.

5 Evaluation

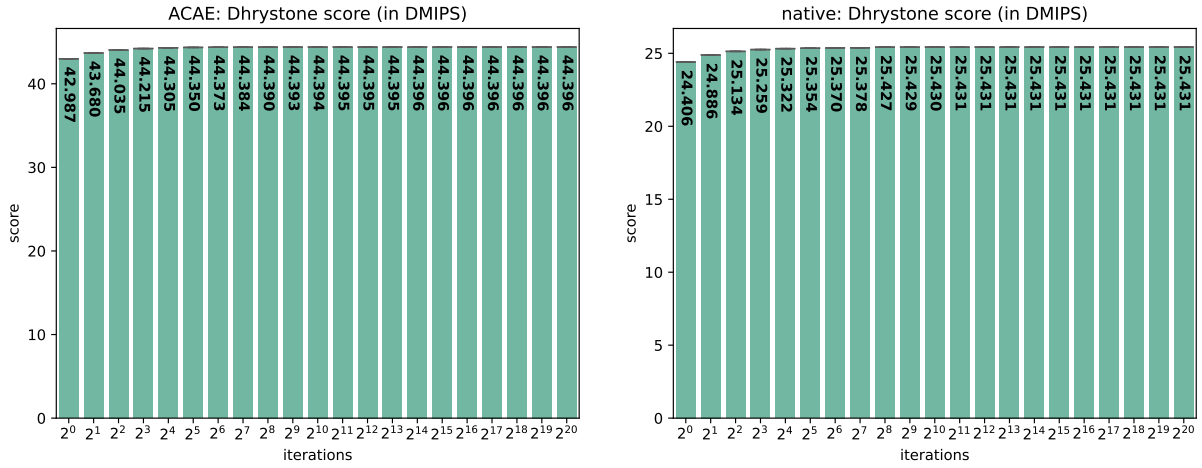
In the following section, we present our results, discuss them, and provide reasons and implications. Lastly, we describe limitations of our work, as well as our emulator, posing opportunities for future work to improve on the state presented today.

5.1 Results

In the following section, we present the performance results of our emulation, as well as the validation on real HW. We measured the performance by running the 2 selected benchmarks,

Dhrystone and CoreMark, with various iteration counts, repeating each run 3 times. The standard deviation between runs of identical configurations is shown as a box plot, while the absolute score results are depicted as bars.
We adjusted the recorded maximum iteration counts between Dhrystone and CoreMark, since both benchmarks differ in time required per iteration significantly. The minimum iteration count is given by 1 for both benchmarks. We used these limits to obtain the largest possible dataset.
All results obtained on our emulator are denoted as *ACAE*, and on the physical MCU as *native*. *Reference* marks performance values given by the MCU vendor.

5.1.1 Dhrystone



(a) Dhrystone score on ACAE

(b) Dhrystone score on native

Figure 1: Dhrystone score

First, we ran Dhrystone on our emulator, starting at 1 iteration, increasing iterations exponentially to base 2 with a maximum iteration count of 2^{20}. The resulting scores are reported in DMIPS over a logarithmic scale of iterations, as shown by Fig. 1a. They start with a score of 42.987 DMIPS at 1 iteration. Following that, there is an increase in score for higher iteration counts, approaching a limit. Starting at 2^{14} iterations, the highest score of 44.396 DMIPS is reached and remains until the maximum iteration count of 2^{20}. The box plot resolving the deviation between the 3 repetitions of identical configuration appears as a line, showing that there is no standard deviation.
Second, we ran Dhrystone on our HW ARM Cortex-M4 board for validation measurements. These scores are then again reported in DMIPS over a logarithmic scale of iterations, as shown by Fig. 1b. They start with a score of 24.406 DMIPS at 1 iteration. Following that, there is an increase in score for higher iteration counts, approaching a limit. Starting at 2^{11} iterations the highest score of 25.431 DMIPS is reached and remains until the maximum iteration count of 2^{20}. The box plot resolving any deviation between our 3 runs of exact equal configuration appears as a line, showing that there is no standard deviation.
Notably, the scores of Dhrystone running on our emulator are higher than on the native HW. This can be seen at 1 iteration, with a score increase of 76.1% on our emulator in reference to the native HW. This slightly decreases for the highest iteration count of 2^{20} at a score increase 74.6% on our emulator in reference to the native HW. Additionally, the observed decrease in performance at 1 iteration amounts to 3.2% of the maximum score at 2^{20} iterations on ACAE and 4.0% on the native HW, respectively.
Fig. 2 explicitly shows the discrepancy in performance, which is measured in DMIPS at 2^{20}

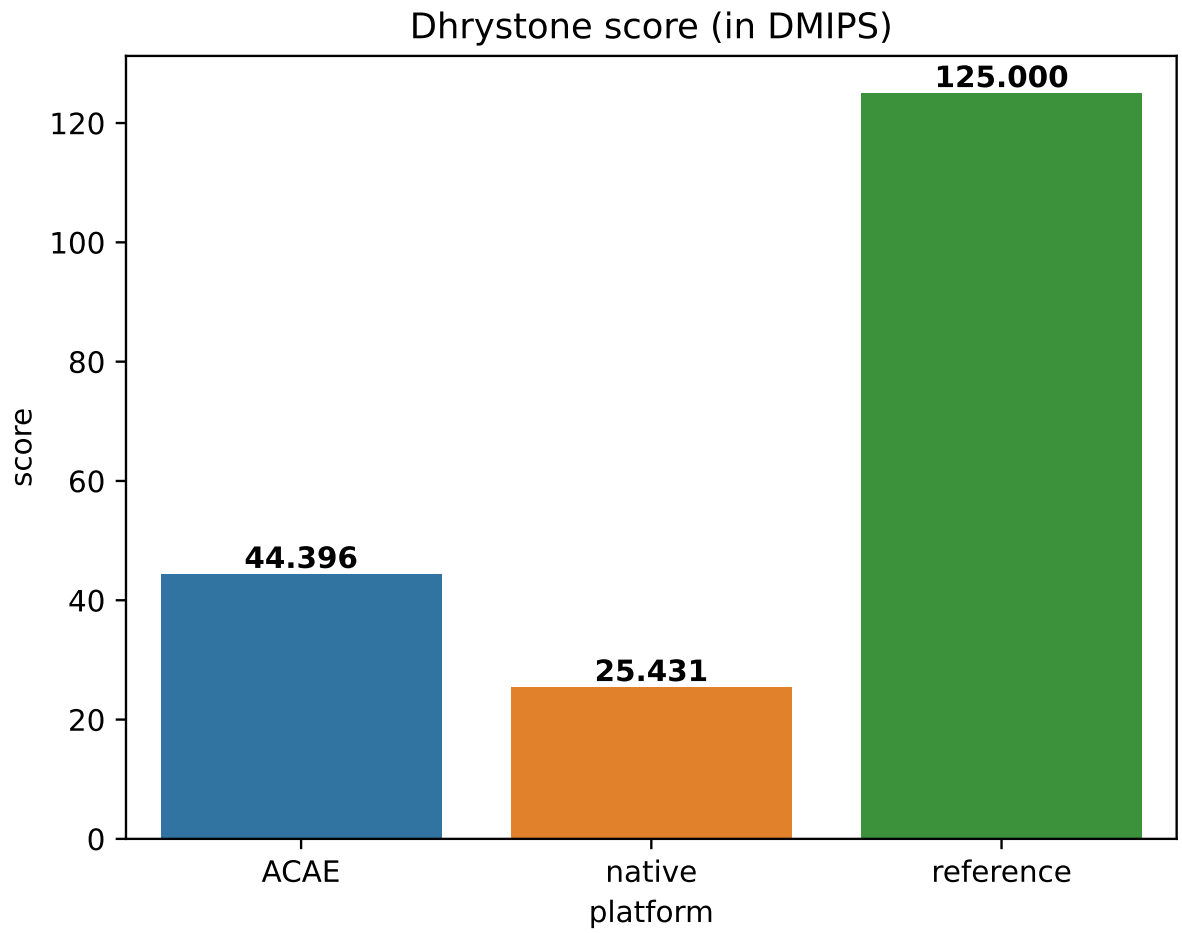
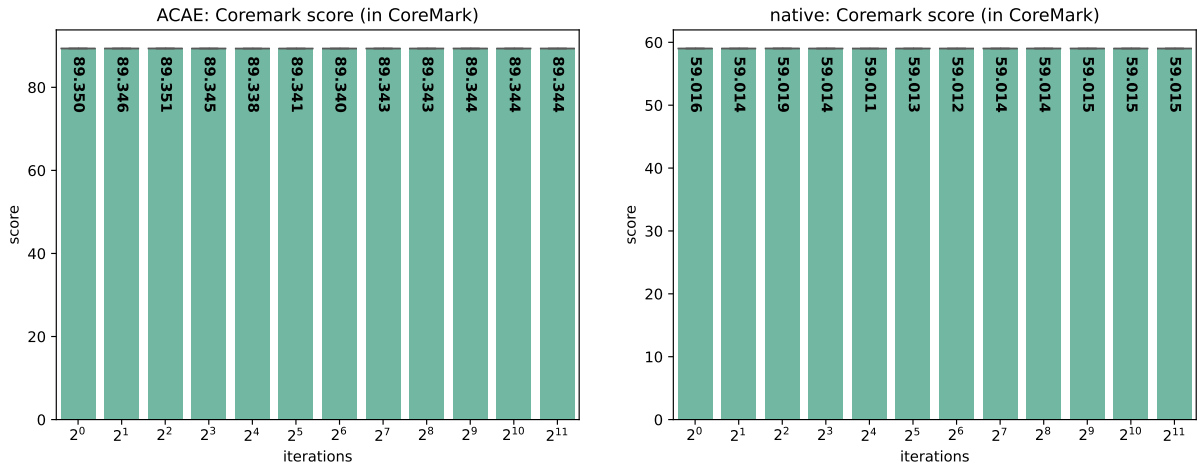


Figure 2: Dhrystone score compared

iterations. It contains the results of our emulator and the real HW for validation. The third bar shows the reference value that was obtained directly from a comparison of ARM Cortex-M chips by ARM [28], as well as the STM32F411xC data sheet of our HW chip. Both claim a score of 125 DMIPS at our model's targeted 100MHz and do not provide any detail on the used compiler, optimizations, or C standard. Both used Dhrystone 2.1, matching our setup in that regard. This provides a proper comparison between our emulator and the real HW, while showing a coarse insight into publicly available vendor performance claims, missing the necessary context.

5.1.2 CoreMark

Following Dhrystone, we run CoreMark on our emulator, starting at 1 iteration, increasing iterations exponentially to base 2 with a maximum iteration count of 2^{11}. The final results are reported in CoreMark over a logarithmic scale of iterations, as shown by Fig. 3a. They start with a score of 89.350 CoreMark at 1 iteration. Following that, there are insignificant deviations at iterations below 2^9. Starting at 2^9 iterations, the score of 89.344 CoreMark remains the same for 2^{10} and 2^{11} iterations. The box plot resolving any deviation between our 3 runs of exact equal configuration appears as a line, showing that there is no standard deviation.
Similarly, as the second step, we ran CoreMark on our HW ARM Cortex-M4 board for validation results. These results are depicted in Fig. 3b. They start with a score of 59.016 CoreMark at 1 iteration. Following that, there is insignificant deviation at iterations below 2^9. Starting at 2^9 iterations, the score of 59.015 CoreMark remains the same for 2^{10} and 2^{11} iterations. The box



(a) CoreMark Score on ACAE

(b) CoreMark Score on native

Figure 3: CoreMark Score

plot resolving any deviation between our 3 runs shows no standard deviation.
Similar to Dhrystone, the scores of CoreMark running on our emulator are higher than on the native HW. This can be seen best at the stable high iteration counts from 2^9 to 2^{11} iterations, with a score increase of 51.4% on our emulator in reference to the native HW.
Fig. 4 explicitly shows the discrepancy in performance, which here is measured in CoreMark at 2^{11} iterations. Similar to Dhrystone, it contains the results of our emulator, as well as the real HW for validation. The third bar shows the reference value that was obtained directly from a comparison table by ARM, which claims a score of 3.42 CoreMark/MHz, resulting in 342 CoreMark at 100MHz [28]. Similarly, there is no detail provided on the used compiler, optimizations, or C standard. This yields a comparison between our emulator and the real HW, while the vendor claim is provided for context.

5.2 Discussion

Overall, our results suggest that emulation of an SM MCU is possible. Additionally, in the setting of a fast emulation using a functional emulator, performance estimations are viable.
We observe that Dhrystone runs 74.6% and CoreMark 51.4% faster on our emulator than on real HW. Since this runtime is not defined by the host execution time of the emulation, but rather by the cycle accounting using our model, we suggest that this discrepancy arises mainly from our simplified cycle model. This aligns with our approach of assuming 1 cycle per instruction by default. This assumption does hold for most instructions, according to the ARM documentation, but does not resolve heavier instructions, such as division or branches. Especially branches, which we assume are quite frequent, will cause a pipeline refill. A pipeline refill affects the next instruction to reach the execution stage 3 cycles after a branch execution, under accounting this instruction by a factor of 3. A worse effect arises using division instructions, which are documented to require 8 to 12 cycles. We suggest that our results reflect this exact deviation, accounting for most instructions properly, but significantly under accounting for some rarer but still frequent instructions. Considering these circumstances and given our performance estimation within the same order of magnitude, we are confident that our emulator could yield sufficiently precise results, provided further adjustments. We suggest that a validation-driven refinement to the cycle model poses a promising subject for future work.
Apart from that, our data implies that our emulator is reproducing HW characteristics in other regards. The HW board running a bare-metal program is fully deterministic, which is matched

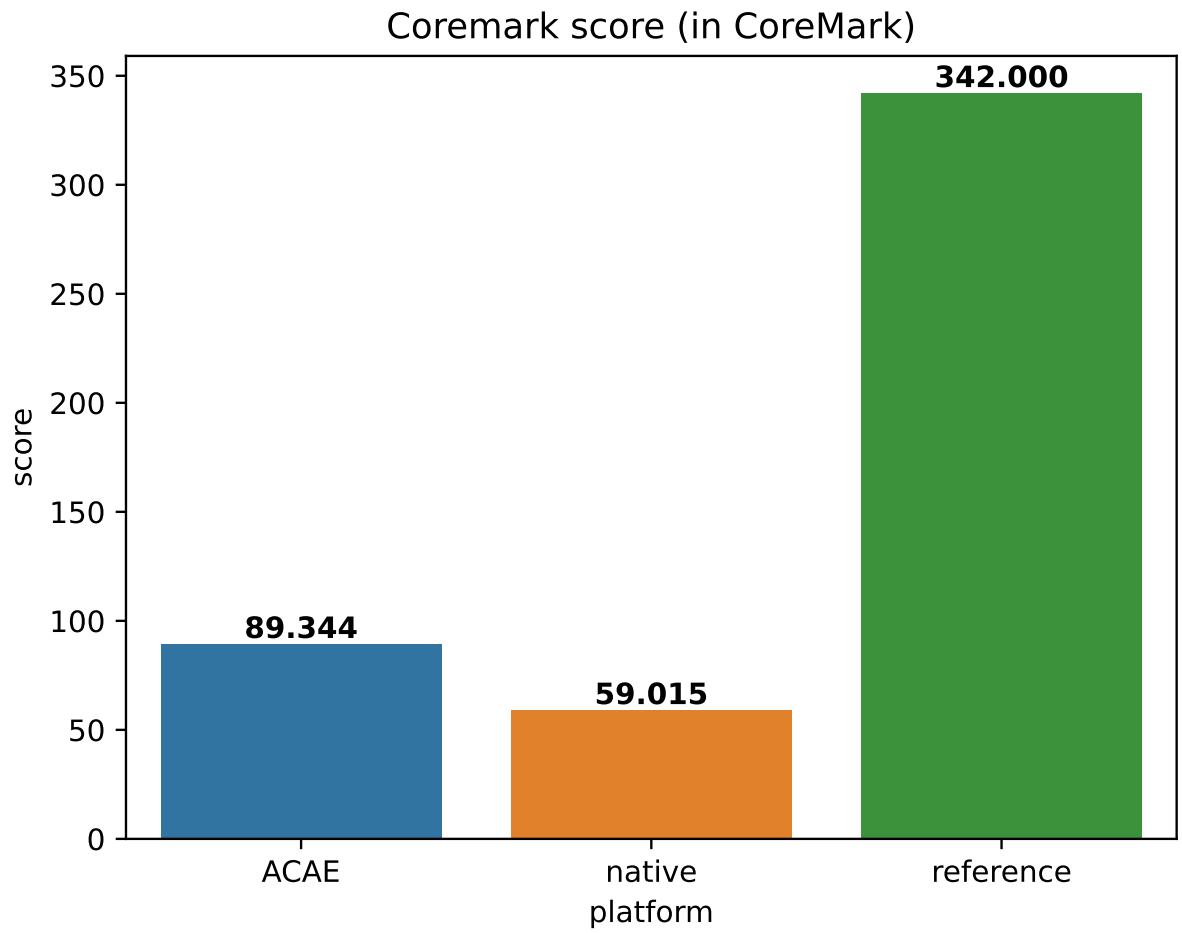


Figure 4: CoreMark score compared

by our emulation. On the one hand, this is necessary for our cycle accounting strategy, but on the other hand, it provides reproducibility posing a solid foundation for the comparability of performance measurements. Another such characteristic is the observed performance decrease of Dhrystone at 1 iteration. While on native HW the decrease amounts to 4.0% of the maximum score, our emulation shows a similar decrease of 3.2%. We observe repeatedly, that our emulation performs better than the real HW, which we attribute to the insufficiently accounted instructions mentioned above. In contrast to Dhrystone, CoreMark does not show such instability at low iterations, but remains stable across iterations with insignificant deviations on both platforms. This may be attributed to the more modern and thoughtful design of CoreMark for benchmarking embedded systems.
Finally, we observe a major discrepancy between our results and our vendor reference values. Our Dhrystone result on our real HW is 79.7% lower in reference to the vendor value. Similarly, the validation CoreMark score is 82.7% lower in reference to the vendor value. We propose that these discrepancies arise from our approach of worst-case performance. We do not make use of compiler optimizations or HW specific implementations of the C standard library. In contrast, we do expect vendors to optimize their performance results as much as possible for marketing reasons. Considering that no context was provided to these reference values, we cannot definitively claim that this is the case, and we solemnly included these values as a comparison to publicly available data.

5.3 Limitations

In the following section, we propose concise limitations of our work, as well as our emulator. We frame these limitations in their context and provide suggestions for improvement.
As described in Sec. 3.1, our emulation builds on publicly available data regarding SM MCU data. We gathered this data from multiple sources, but found it to be rather scarce. This limits our dataset and does not allow for a statistical evaluation of a broad range of SMs. We use the available data to create our single SM MCU model, based on chips suggested for electrical metering. This excludes SMs for different resources, as well as different device classes, such as high-end or low-end, as originally considered. We propose that this could be improved either by direct cooperation with SM vendors or a general open source shift of the SM market. In our opinion, both seem unlikely unless properly incentivized.
The limitation to 2 benchmarks arises from our time constraint, as well as from compatibility and utility reasons. We focus on Dhrystone and CoreMark, as explained in Sec. 2.4 for their broad use in embedded systems. Both benchmarks have publicly available performance data, which we consider for validation. Implementing other benchmarks does not promise to provide similar reference data. We do suggest however, that other benchmarks would provide meaningful validation data if validation runs are recorded on native HW, as we do with Dhrystone and CoreMark. We propose that implementing a broader range of benchmarks followed by appropriate adjustments to the cycle model poses the best next step to improving the emulator.
In the process of constructing our emulator, we assume multiple aspects to keep our work feasible. First, our emulator is tailored to our model and methodology.
Therefore, it supports exclusively the ARM Cortex-M4 chip family. Our modifications to the QEMU board do not natively translate to other chips or architectures and would have to be reconsidered and reimplemented accordingly.
The build system is based on a bare-metal architecture. While an OS could be more user-friendly, we do not provide the tool chain for building and flashing non-monolithic binaries.
We provide a startup system and selected system functions. Any future software requirements, such as extended STD support, may need dedicated implementation.
For our explicit board, presented in Sec. 3.3, any HW validation data reporting is done by serial communication using another chip. In order to use the onboard USB port, a USB stack would have to be implemented on bare-metal. We suggest that integrating a Hardware-Abstraction-Layer driver into our bare-metal setup could provide the fastest way to reach native USB functionality, while imposing some overhead.
Second, we constrain binaries to require function symbols. This limitation arises from QEMU's plugin implementation, focusing more on pure instrumentation and less on synchronized host-to-guest interaction. Function symbols currently allow for faster emulation performance on the host using our QEMU plugin. We suggest that the use of compiler optimization and, therefore, omission of symbols would provide a larger dataset for validation and potential cross-referencing to public vendor data. However, this would slow emulation speed or require major changes to the QEMU source code.
Lastly, our emulation explicitly implements the MCU. While peripherals such as ADCs or network interface controllers would be desirable for a complete SM emulation, their realization would require major modifications to the QEMU source code and is beyond the scope of our work.
Our emulator is tailored to our model as described in Sec 3.1.3 and requires some improvement on the cycle model, but overall mimics SM MCU characteristics in a promising manner.

6 Conclusion

We suggest that SMs are expected to play an increasingly active role in future SGs, potentially serving as edge-computing devices that support advanced functionality beyond measurement and reporting. However, research on such extensions is hindered by the proprietary nature of commercially deployed SMs and the lack of accessible development and evaluation environments. To address this challenge, this work introduced ACAE, a HW-independent SM emulation approach based on a representative SM MCU model derived from publicly available specifications. To realize ACAE, QEMU's ARM Cortex-M4 board was extended with cycle-accounting capabilities. The resulting emulator aims to facilitate the development and performance-oriented evaluation of novel SM applications without requiring access to proprietary HW.
The proposed approach was evaluated using the Dhrystone and CoreMark benchmarks. Relative to the reference HW platform, Dhrystone executed 74.6% faster on the emulator, while CoreMark executed 51.4% faster, respectively. Despite these deviations in absolute score, ACAE successfully reproduced the observed performance trends of both benchmarks. In particular, Dhrystone exhibited increasing benchmark scores with increasing iteration counts on both platforms, while CoreMark remained largely stable across all evaluated iteration counts. These results indicate that the implemented cycle-accounting approach captures relevant execution characteristics and provides a promising basis for performance estimation. Although the cycle model requires further refinement to improve timing accuracy.
Several limitations remain. The current cycle model does not yet account sufficiently for all instruction classes and execution effects. In particular, the influence of pipeline refills, division instructions, and other micro-architectural effects should be investigated to improve the accuracy of cycle estimation. Furthermore, the representative MCU model was derived exclusively from publicly available data sheets of MCUs proposed for smart metering applications, as detailed documentation of commercial SM HW is generally unavailable. The evaluation was limited to two benchmark suites and a single ARM Cortex-M4 architecture. In addition, the current implementation requires binaries to be compiled with symbols included and without compiler optimizations, restricting the range of software that can presently be evaluated.
Future work will therefore focus on refining the cycle model to improve timing accuracy and extending the validation using a broader set of benchmark applications. A larger validation dataset will provide deeper insight into the strengths and limitations of the proposed approach and support the calibration of the cycle-accounting mechanism. Furthermore, enabling support for optimized binaries would significantly increase the practical applicability of ACAE and allow the evaluation of software under conditions that more closely resemble real-world deployments. Through these extensions, ACAE can evolve into a more accurate and versatile tool for researching future SM functionality and SG applications.

References

[1] Arman Goudarzi et al. "A Survey on IoT-Enabled Smart Grids: Emerging, Applications, Challenges, and Outlook". In: *Energies* 15.19 (2022). ISSN: 1996-1073. DOI: 10.3390/en15196984. URL: https://www.mdpi.com/1996-1073/15/19/6984.

[2] Gouri R. Barai, Sridhar Krishnan, and Bala Venkatesh. "Smart metering and functionalities of smart meters in smart grid - a review". In: *2015 IEEE Electrical Power and Energy Conference (EPEC)*. 2015, pp. 138–145. DOI: 10.1109/EPEC.2015.7379940.

[3] BMJV. *Gesetz über den Messstellenbetrieb und die Datenkommunikation in intelligenten Energienetzen 1 (Messstellenbetriebsgesetz - MsbG) § 52 Allgemeine Anforderungen an die Datenkommunikation; Anonymisierung und Pseudonymisierung*. https://www.gesetze-im-internet.de/messbg/__52.html. Accessed: 2025-12-02. 2023.

[4] BMJV. *Gesetz über den Messstellenbetrieb und die Datenkommunikation in intelligenten Energienetzen 1 (Messstellenbetriebsgesetz - MsbG) § 55 Messwerterhebung Strom*. https://www.gesetze-im-internet.de/messbg/__55.html. Accessed: 2025-12-02. 2023.

[5] Muhammad Rizwan Asghar et al. "Smart Meter Data Privacy: A Survey". In: *IEEE Communications Surveys & Tutorials* 19.4 (2017), pp. 2820–2835. DOI: 10.1109/COMST.2017.2720195.

[6] Todd Baumeister. *Literature Review on Smart Grid Cyber Security*. https://csdl.ics.hawaii.edu/techreports/2010/10-11/10-11.pdf. Accessed: 2025-12-02. 2010.

[7] Jixuan Zheng, David Wenzhong Gao, and Li Lin. "Smart Meters in Smart Grid: An Overview". In: *2013 IEEE Green Technologies Conference (GreenTech)*. 2013, pp. 57–64. DOI: 10.1109/GreenTech.2013.17.

[8] Andrew S Tanenbaum. *Structured computer organization*. Pearson Education India, 2016.

[9] Sergey Lyubka. *A bare-metal programming guide*. https://developer.arm.com/community/arm-community-blogs/b/internet-of-things-blog/posts/a-bare-metal-programming-guide. Accessed: 2026-05-10.

[10] Arm Limited. *Arm Cortex-M4 Processor Technical Reference Manual*. https://developer.arm.com/documentation/100166/0001. Accessed: 2026-05-01.

[11] Software Freedom Conservancy. *QEMU*. https://www.qemu.org/. Accessed: 2025-12-01.

[12] gem5 Project management committee. *gem5*. https://www.gem5.org/. Accessed: 2025-12-01.

[13] antmicro. *renode*. https://renode.io/. Accessed: 2026-04-20.

[14] Ayaz Akram and Lina Sawalha. "A Survey of Computer Architecture Simulation Techniques and Tools". In: *IEEE Access* 7 (2019), pp. 78120–78145. DOI: 10.1109/ACCESS.2019.2917698.

[15] Igor Böhm, Björn Franke, and Nigel Topham. "Cycle-accurate performance modelling in an ultra-fast just-in-time dynamic binary translation instruction set simulator". In: *2010 International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation*. 2010, pp. 1–10. DOI: 10.1109/ICSAMOS.2010.5642102.

[16] Fabrice Bellard et al. "QEMU, a fast and portable dynamic translator." In: *Usenix ATC, Freenix Track*. 2005, pp. 41–46.

[17] Alan R Weiss. "Dhrystone benchmark". In: *History, Analysis„„Scores "and Recommendations, White Paper, ECL/LLC* (2002).

[18] EEMBC. *EEMBC*. https://www.eembc.org/. Accessed: 2025-12-18.

[19] Chien-I Lee et al. "Iotbench: A Benchmark Suite for Intelligent Internet of Things Edge Devices". In: *2019 IEEE International Conference on Image Processing (ICIP)*. 2019, pp. 170–174. DOI: 10.1109/ICIP.2019.8802949.

[20] Simin Chen et al. "IoTBench: A data centrical and configurable IoT benchmark suite". In: *BenchCouncil Transactions on Benchmarks, Standards and Evaluations* 2.4 (2022), p. 100091. ISSN: 2772-4859. DOI: https://doi.org/10.1016/j.tbench.2023.100091. URL: https://www.sciencedirect.com/science/article/pii/S277248592300008X.

[21] Naif Saleh Almakhdhub et al. "BenchIoT: A Security Benchmark for the Internet of Things". In: *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2019, pp. 234–246. DOI: 10.1109/DSN.2019.00035.

[22] Jakob Fregin. *ACAE*. https://github.com/cheater78/acae. Accessed: 2026-06-12.

[23] Reinhold P. Weicker. "Dhrystone: a synthetic systems programming benchmark". In: *Commun. ACM* 27.10 (Oct. 1984), pp. 1013–1030. ISSN: 0001-0782. DOI: 10.1145/358274.358283. URL: https://doi.org/10.1145/358274.358283.

[24] Diana Gutierrez et al. "Virtualization and Validation of Emulated STM-32 Blue Pill Using the QEMU Open-Source Framework". In: *2023 11th International Symposium on Digital Forensics and Security (ISDFS)*. 2023, pp. 1–6. DOI: 10.1109/ISDFS58141.2023.10131693.

[25] Shay Gal-On and Markus Levy. "Exploring coremark a benchmark maximizing simplicity and efficacy". In: *The Embedded Microprocessor Benchmark Consortium* 6.23 (2012), p. 87.

[26] TOSUNlux. *Top 10 Smart Meter Suppliers in 2024*. https://www.tosunlux.eu/blog/top-smart-meter-supplier/. Accessed: 2025-11-25.

[27] Blackridge Research & Consultance. *Global Top 10 Smart Meter manufacturers*. https://www.blackridgeresearch.com/blog/list-of-global-top-smart-electric-electricity-gas-water-meter-companies-manufacturers-makers-suppliers-in-the-world. Accessed: 2025-12-03.

[28] Arm Limited. *Arm Cortex-M Processor Comparison Table*. https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Cortex-A%20R%20M%20datasheets/Arm%20Cortex-M%20Comparison%20Table_v3.pdf. Accessed: 2026-05-18.